

Grant Types Demo

Assumption: By default, realm is created with name (eazybankdev)

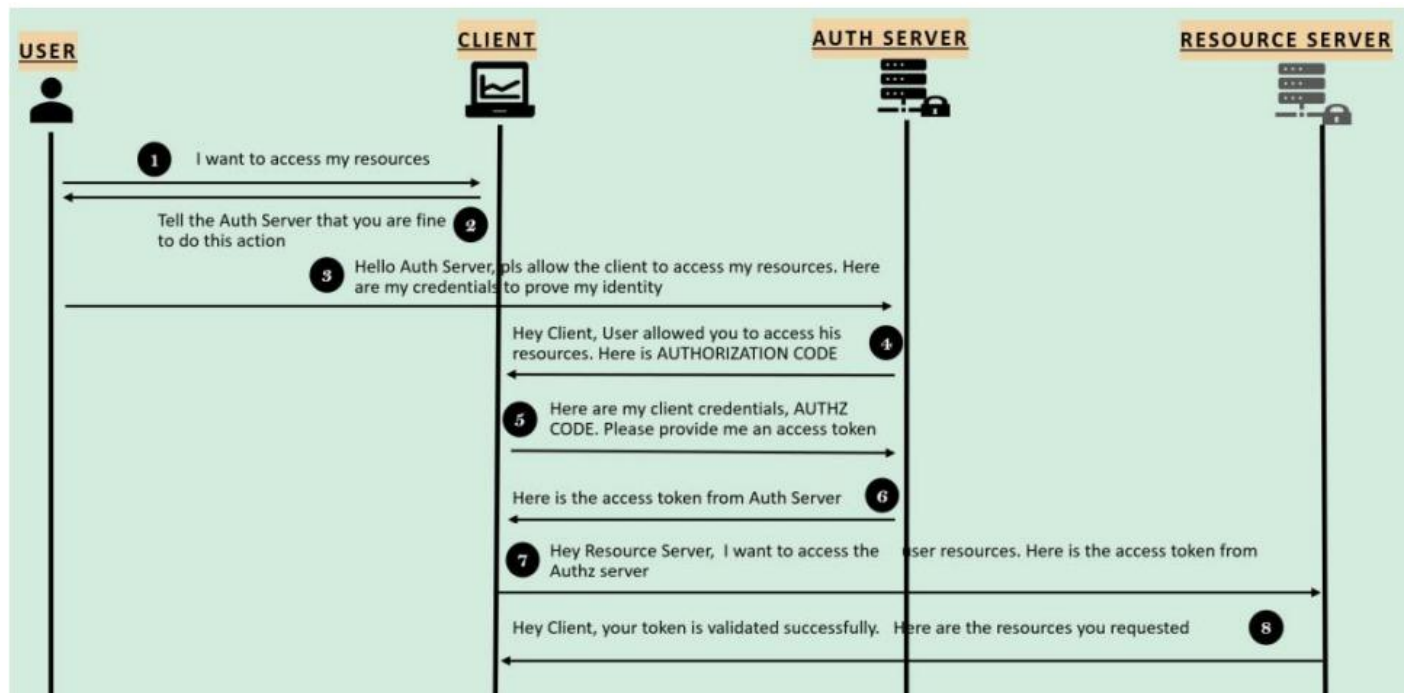
Resource Owner / End User – happy@example.com

Client – Postman

Auth Server – KeyCloak (deployed in local docker)

Resource Server - NA

Authorization Code Grant Type



Step 1: Client Registration – Post registration, we will have ClientID & ClientSecret (optional)

Clients > Client details

eazybankauthorizationcode

OpenID Connect

Clients are applications and services that can request authentication of a user.

Settings

Roles

Client scopes

Sessions

Advanced

Events

General settings

Client ID * ⓘ

eazybankauthorizationcode

Name ⓘ

eazybankauthorizationcode

Description ⓘ

Authorization Code Grant type demo

Always display in UI ⓘ

Off

Capability config

Client authentication ⓘ

Off

Authorization ⓘ

Off

Authentication flow

☒ Standard flow ⓘ

☐ Direct access grants ⓘ

☐ Implicit flow ⓘ

☐ Service accounts roles ⓘ

☐ Standard Token Exchange ⓘ

☐ OAuth 2.0 Device Authorization Grant ⓘ

☐ OIDC CIBA Grant ⓘ

PKCE Method ⓘ

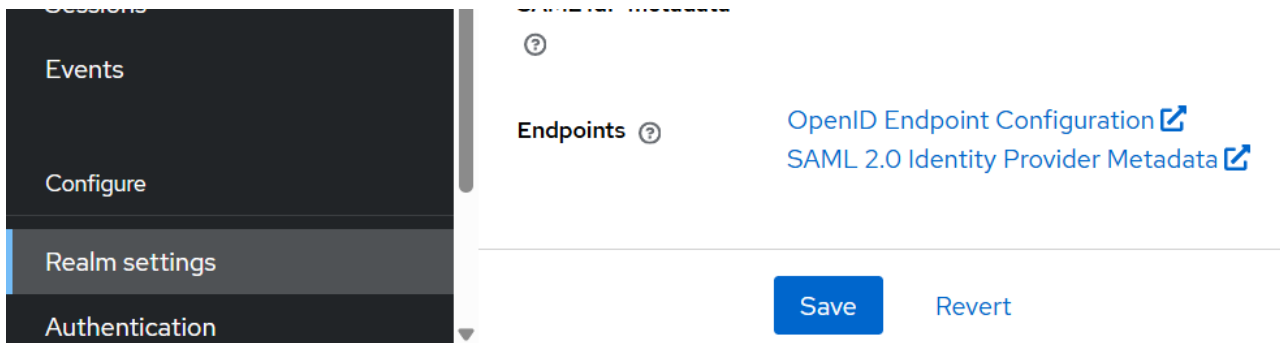
Choose...

ClientID – eazybankauthorizationCode

ClientSecret – (Optional)

Step 2: Get token URI & Auth URL from below endpoint (under Realm setting – make sure you’ve selected the correct realm i.e., eazybankdev

OpenID Configuration - <http://localhost:8180/realms/eazybankdev/.well-known/openid-configuration>



Step 3: Add all these configurations to Postman to generate Auth Code

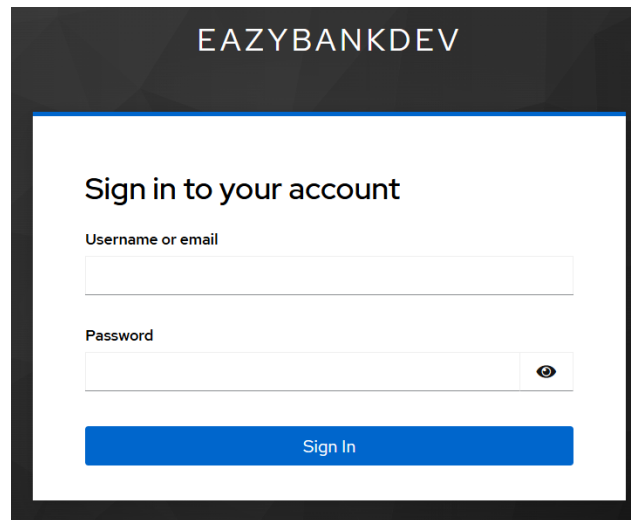
Auth URL – <http://localhost:8180/realms/eazybankdev/protocol/openid-connect/auth>

Access Token URL - <http://localhost:8180/realms/eazybankdev/protocol/openid-connect/token>

State – Any Random value or Empty

Step 4: Click on Get New Access Token → Redirected to below Authorization URL

```
http://localhost:8180/realms/eazybankdev/protocol/openid-connect/auth?
response_type=code
&client_id=eazybankauthorizationcode
&state=2fc61105-40ba-4b5f-8983-ec764ec4e437
&scope=openid%20email
&redirect_uri=https%3A%2F%2Foauth.pstmn.io%2Fv1%2Fcallback
```



Step 5: On Sign in click, below authenticate with (302 status) will be invoked & User “happy” will be redirected back to the client, & we can notice a few additional query parameters in the URL of callback API (specifically **state** & **code**)

Callback URL - <https://oauth.pstmn.io/v1/callback>

Name	✕	Headers	Payload	Preview	Response	Initiator	Timing
authenticate?session_...							
callback?state=2fc611...							
favicon.ico							
			▼ Query String Parameters	View source	View URL-encoded		
			state	2fc61105-40ba-4b5f-8983-ec764ec4e437			
			session_state	64a30195-9148-4053-b96b-b075fa8e9b1b			
			iss	http://localhost:8180/realms/eazybankdev			
			code	a720eb40-c4f4-48c5-8193-9cfa2a9527a1.64a30195-9148-4053-b96b-b075fa8e9b1b.ce16b28a-07e9-476a-8b5e-62a805aff631			

```
https://oauth.pstmn.io/v1/callback?
state=2fc61105-40ba-4b5f-8983-ec764ec4e437
&code=a720eb40-c4f4-48c5-8193-9cfa2a9527a1.64a30195-9148-4053-b96b-b075fa8e9b1b.ce16b28a-07e9-476a-8b5e-62a805aff631
&session_state=64a30195-9148-4053-b96b-b075fa8e9b1b
&iss=http%3A%2F%2Flocalhost%3A8180%2Frealms%2Feazybankdev
```

Note:

- We need to first verify that the “state” parameter (in the redirect) matches the value stored in this user’s session so that we protect against CSRF.
- Depending on how we’ve stored the “state” parameter (in a cookie, session or some other way), verify that it matches the state that we originally included in Step 3 & 4.

Step 6: Exchange the Authorization code: If state matches, we’re ready to exchange the Authorization Code for an Access Token.

(Step 3 part of OpenID Config):

Access Token URL - <http://localhost:8180/realms/eazybankdev/protocol/openid-connect/token>

The Client (e.g., Postman) will build a POST request to the token endpoint:

Online Find and replace Console

▼ POST http://localhost:8180/realms/eazybankdev/protocol/openid-connect/token

► Network

► Request Headers

▼ Request Body

```
grant_type: "authorization_code"
code: "a720eb40-c4f4-48c5-8193-9cfa2a9527a1.64a30195-9148-4053-b96b-b075fa8e9b1b.ce16b28a-07e9-476a-8b5e-62a805aff631"
redirect_uri: "https://oauth.pstmn.io/v1/callback"
client_id: "eazybankauthorizationcode"
client_secret: ""
```

Note: The Client's credentials are included in the POST body in this example. Other Authorization servers may require that the credentials are sent as HTTP Basic Authentication header.

OAuth 2.0 Authorization Code Flow conclusions

Step 1 – Build the authorization URL & redirect the user to the Authorization Server.

Step 2 – After the user is redirected back to the Client, verify the state matches

Step 3 – Exchange the Authorization code for an Access Token

< Response Body

{ } JSON ▾

```
1 {
2   "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpzZW50L3N1bWVhcnR5Iiwia2kiOiA6ICJhcGE4dFpVa3RVRVhWRWVoY1AxWFVHZAkxwV1Zl
   eyJleHAiOiJlNTM0ODM4NDZksImldCI6MTc1Mzg4MzU3OSwiYXV0aF90aW1lIjoxNzUzODgzNTc4LjQqdGkiOiJvbnJ0YWM6MGi0z
   y9lYXp5YmFua2RldiIsImF1IjY2Y291bnQlLCJzdWIiOiJjMGFjYzhlNS1kOWF1LTQ1YzktOTYzNC0wMTJkMjkzMWl0YTEiLC
   00MDUzLWl5NmItYjA3NWZhOGU5YjFiIiwiaWVhcnR5IjoimSI6ImF1IjY2Y291bnQlLCJzdWIiOiJjMGFjYzhlNS1kOWF1LTQ1YzktOTYzNC0wMTJkMjkzMWl0YTEiLC
   7ImFjY291bnQlOisicm9sZXMiOiIsbWVhcnR5IiwiaWVhcnR5IjY2Y291bnQlLCJzdWIiOiJjMGFjYzhlNS1kOWF1LTQ1YzktOTYzNC0wMTJkMjkzMWl0YTEiLC
   IEV4YmVhcnR5IiwiaWVhcnR5IjY2Y291bnQlLCJzdWIiOiJjMGFjYzhlNS1kOWF1LTQ1YzktOTYzNC0wMTJkMjkzMWl0YTEiLC
   LoJlVZdUhm8g1zi731DvA5V16T11Q_WA73336s_NhOq06CXuBKQj630bmvxr7Um7SDHTQX6GrjAxbA16r0dGAKflqmNqle3vgXLWt
   ioIA90sMA8637VE19bvFNHj-ks4Q2aC0mJHqNcANlv8wKJohwn05p8AUw8_Be71_C8hFwhxg-7hQ88Yt9c4psiAZ_soE6RgQ0D95J
3   "expires_in": 3600,
4   "refresh_expires_in": 1800,
5   "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZW50L3N1bWVhcnR5Iiwia2kiOiA6ICJhcGE4dFpVa3RVRVhWRWVoY1AxWFVHZAkxwV1Zl
   eyJleHAiOiJlNTM0ODM4NDZksImldCI6MTc1Mzg4MzU3OSwiYXV0aF90aW1lIjoxNzUzODgzNTc4LjQqdGkiOiJvbnJ0YWM6MGi0z
   2NhbGhvc3Q6ODE4MjYyZWZlbnR5IiwiaWVhcnR5IjY2Y291bnQlLCJzdWIiOiJjMGFjYzhlNS1kOWF1LTQ1YzktOTYzNC0wMTJkMjkzMWl0YTEiLC
   kxNDgtNDA1My1iOTZiLWlWbWVhcnR5IiwiaWVhcnR5IjY2Y291bnQlLCJzdWIiOiJjMGFjYzhlNS1kOWF1LTQ1YzktOTYzNC0wMTJkMjkzMWl0YTEiLC
   ksG_2eknBxt61Zx0MF3a4mwobTYh00S0o1YnZjJbnmYKMit9PV_Xcnd1Kc17IUqx8_Rdw0bVgG0ydY6G4iqBg",
6   "token_type": "Bearer",
7   "id_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpzZW50L3N1bWVhcnR5Iiwia2kiOiA6ICJhcGE4dFpVa3RVRVhWRWVoY1AxWFVHZAkxwV1Zl
   eyJleHAiOiJlNTM0ODM4NDZksImldCI6MTc1Mzg4MzU3OSwiYXV0aF90aW1lIjoxNzUzODgzNTc4LjQqdGkiOiJvbnJ0YWM6MGi0z
   W5rZGV2IiwiaWVhcnR5IjY2Y291bnQlLCJzdWIiOiJjMGFjYzhlNS1kOWF1LTQ1YzktOTYzNC0wMTJkMjkzMWl0YTEiLC
   k1LTkxNDgtNDA1My1iOTZiLWlWbWVhcnR5IiwiaWVhcnR5IjY2Y291bnQlLCJzdWIiOiJjMGFjYzhlNS1kOWF1LTQ1YzktOTYzNC0wMTJkMjkzMWl0YTEiLC
   hcHB5QGV4YmVhcnR5IjY2Y291bnQlLCJzdWIiOiJjMGFjYzhlNS1kOWF1LTQ1YzktOTYzNC0wMTJkMjkzMWl0YTEiLC
   j84wv3Q2kb83uy51YGCfr1j5r_U7VVB88Dmch8NVJz-GC0twpJNKds1BtgNCCGXivhXmJx_k_7QVNXVnNcHYFctnkFyus4HX5sa/
   twGUhLLakhovslzdWx88Td106DLjiH3nygXK3EK35yIq1PgW4nkgJBpvg8U8LtB3PdH_Cn6uJffEHFVKM4Rrf007h78G3A-8DCuoj
8   "not-before-policy": 0,
9   "session_state": "64a30195-9148-4053-b96b-b075fa8e9b1b",
10  "scope": "openid profile email"
11 }
```

Note: We can use <https://www.jwt.io/> to decode the all types of tokens.

Access Token

- An Access Token is a security token (JWT) issued by an Authorization Server as part of an OAuth 2.0 flow.
- It contains information about the user & the resources (Web APIs & protected resources) for which the token is intended.
- When an application needs to access protected resources, it includes the Access Token in the HTTP Authorization header as a Bearer token.

Refresh Token

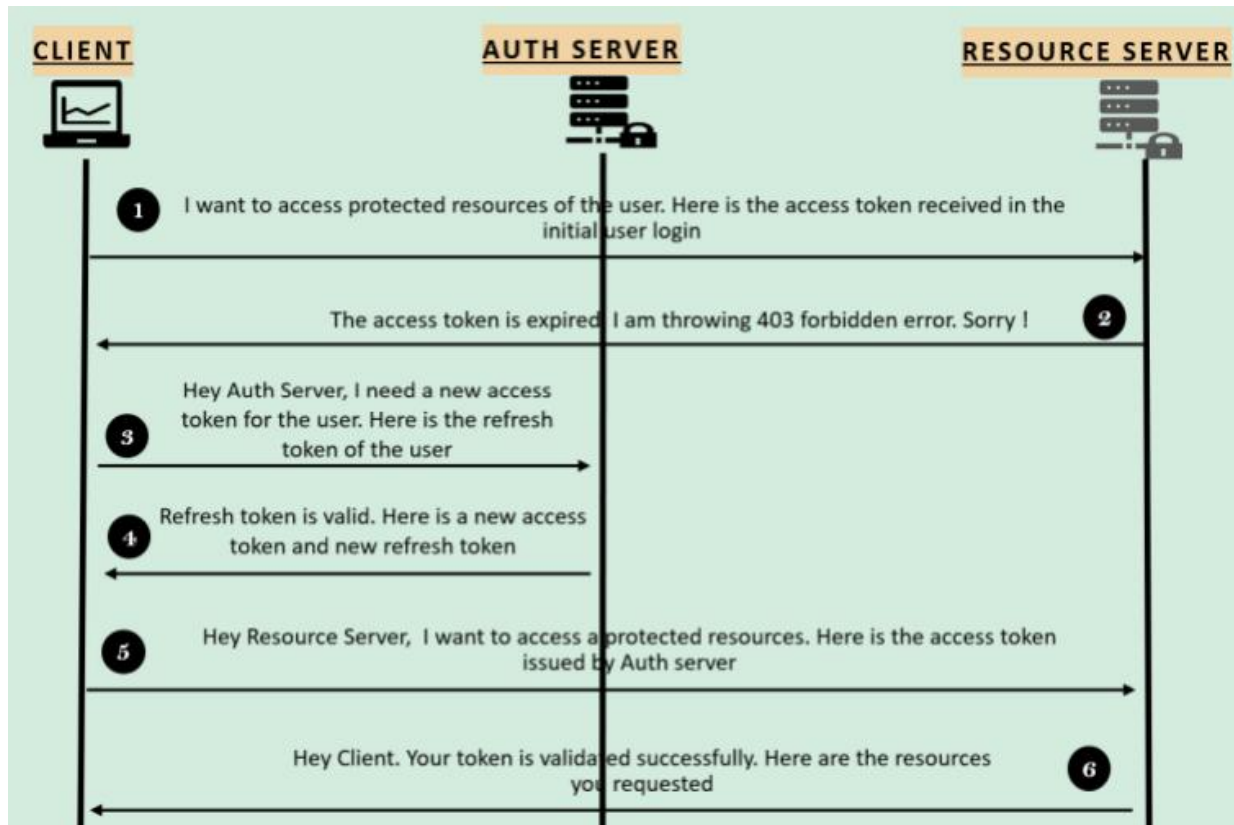
- A Refresh token is a special token (JWT) that can be used to obtain new access tokens or ID tokens without requiring the user to re-authenticate.
- This is particularly useful for maintaining a seamless User experience in applications where users need continuous access over extended periods.
- By leveraging Refresh Token Grant Type, the access token will be regenerated.

Id Token

- An ID token is a JSON Web Token (JWT) that contains information about the user such as their unique identifier, name, email & other profile information.
- It is intended to be read by the Client application & is not meant for securing API requests. The ID token allows the client to verify the user's identity & obtain basic profile information.
- While Access Tokens are used to access protected resources & Refresh tokens are used to obtain new Access Tokens, the ID Tokens are typically used to cache user information on the client side, reducing the need to make additional requests to the Authorization Server for user data.

Refresh Token Grant Type

- The Refresh Token Grant Type is used by Clients to exchange a Refresh Token for an Access Token when Access Token has expired.
- This grant type is supported for specific grant types such as Authorization Code, Resource Owner Password Credentials.
- The Client can make a POST request to the token endpoint & the response will include a New Access Token & optionally a New Refresh Token. Refer below:



▼ POST http://localhost:8180/realms/eazybankdev/protocol/openid-connect/token

► Network

► Request Headers

▼ Request Body

- ▶ Response Headers

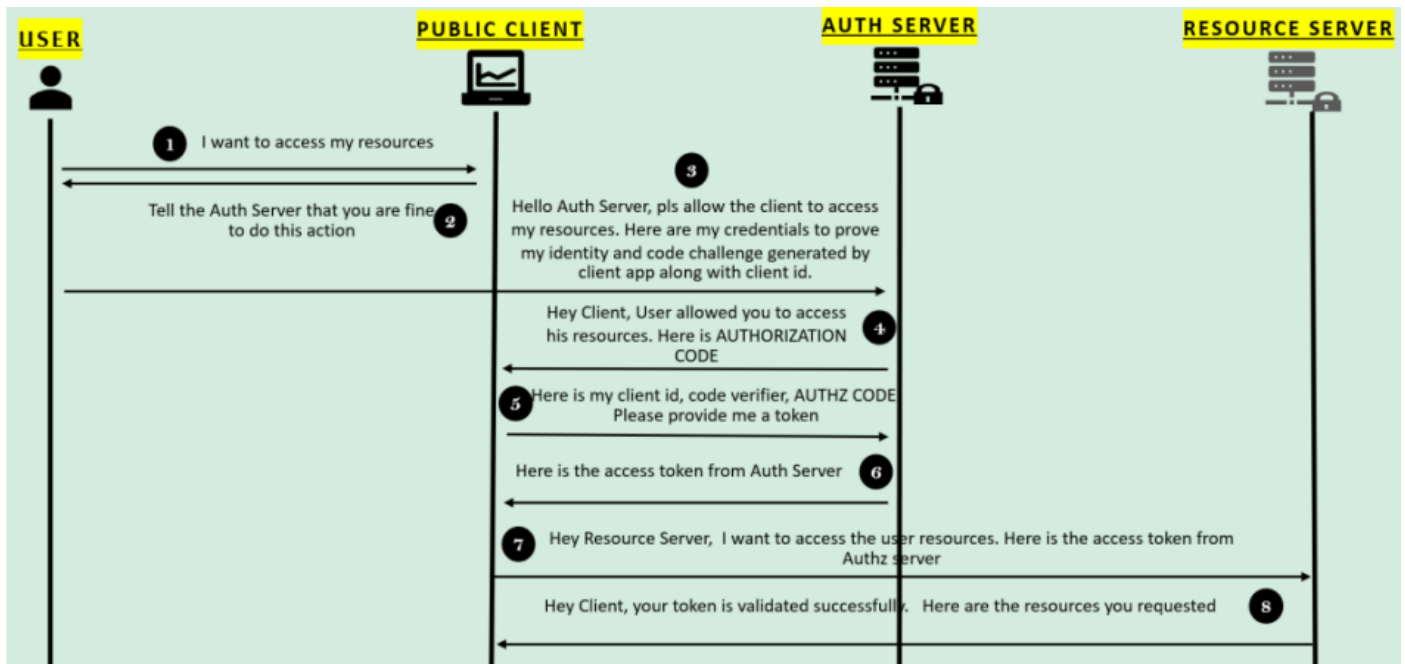
< Response Body

{ }

JSON

[illegible]

Authorization Code with PKCE (Proof Key for Code Exchange)



Step 1: Client Registration – Post registration, we will have ClientID, ClientSecret (optional), PKCE Method as S256

eazybankpkce

OpenID Connect

Clients are applications and services that can request authentication of a user.

Settings

Roles

Client scopes

Sessions

Advanced

Events

General settings

Client ID *

eazybankpkce

Name

eazybankpkce

Description

Authorization Code with PKCE (Proof Key for Code Exchange)

Always display in UI

Off

Capability config

Client authentication

Off

Authorization

Off

Authentication flow

☒ Standard flow

☐ Direct access grants

☐ Implicit flow

☐ Service accounts roles

☐ Standard Token Exchange

☐ OAuth 2.0 Device Authorization Grant

☐ OIDC CIBA Grant

PKCE Method

S256

ClientID – eazybankpkce

Step 2: Create a code verifier & Challenge

- Before redirecting the end user to the Authorization server, the Client first generates a Secret Code Verifier & Challenge.
- The Code verifier is a cryptographically random string using characters A-Z, a-z, 0-9 & the punctuation characters (hyphen, period, underscore & tilde ~)
- Client uses the generated Code verifier to create the Code challenge.
- For devices that can perform a SHA256 hash, the code challenge is a BASE64-URL-encoded string of the SHA256 hash of the code verifier. Otherwise, the same verifier string can be used as the Code Challenge.

Generate Code Verifier

jT684As258C0cJA-ohmGDfmDQhTscilUqqoilBMzziI4_9Rz

Generate Code Challenge

`base64url(sha256(code_verifier))`

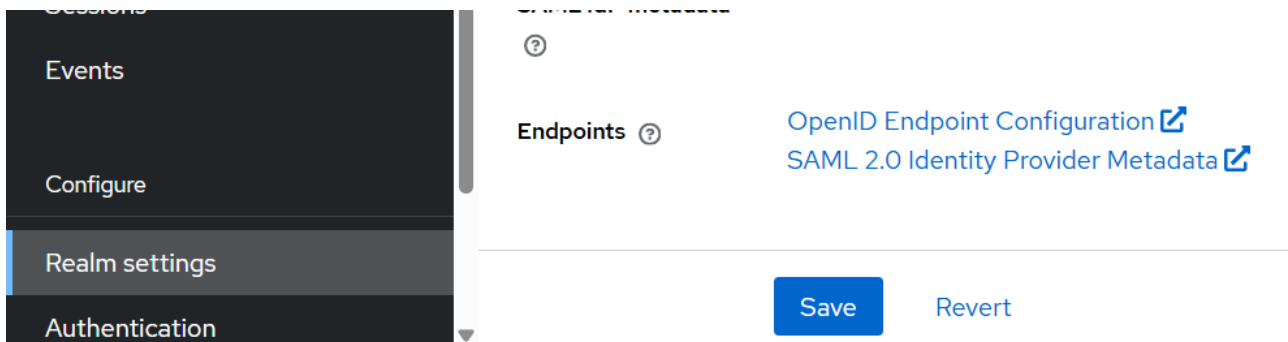
bCTy33EcEIBZKfETF7u2FbrxeIkvhR1Kx0RM1uLo1zw

The client needs to store the `code_verifier` for later use. We will store it in a cookie for this demo.

- For Postman, we can keep it empty. By default, postman will take care of it.

Step 3: Get token URI & Auth URL from below endpoint (under Realm setting – make sure you’ve selected the correct realm i.e., eazybankdev)

OpenID Configuration - <http://localhost:8180/realms/eazybankdev/.well-known/openid-configuration>



Step 4: Add all these configurations to Postman to generate Auth Code

Auth URL – <http://localhost:8180/realms/eazybankdev/protocol/openid-connect/auth>

Access Token URL - <http://localhost:8180/realms/eazybankdev/protocol/openid-connect/token>

Code Challenge Method – SHA-256

Code Verifier – Generated in Step 2

Scope – openid email

State – Any Random value or Empty

Params • **Authorization** • Headers (9) • Body • Scripts • Settings

Auth Type

The authorization data will be automatically generated when you send the request. Learn more about [OAuth 2.0](#) authorization.

Add authorization data to

Grant type

Callback URL ⓘ

☒ Authorize using browser

Auth URL ⓘ

Access Token URL ⓘ

Client ID ⓘ

Client Secret ⓘ

Code Challenge Method ⓘ

Code Verifier ⓘ

Scope ⓘ

State ⓘ

Client Authentication ⓘ

Step 5: Click on Get New Access Token → Redirected to below Authorization URL

```
http://localhost:8180/realms/eazybankdev/protocol/openid-connect/auth?
response_type=code
&client_id=eazybankpkce
&scope=openid%20email
&redirect_uri=https%3A%2F%2Foauth.pstmn.io%2Fv1%2Fcallback
&code_challenge=mimgiPU2WNQwA3GTh233jEhfKoittKwOvOMrfKzC31Y
&code_challenge_method=S256
```

EAZYBANKDEV

Sign in to your account

Username or email

Password

👁

Sign In

Note: The client includes the code_challenge parameter in the request, which the authorization server will store & compare later during the code exchange step.

Step 6: On Sign in click, below authenticate with (302 status) will be invoked & User “happy” will be redirected back to the client, & we can notice a few additional query parameters in the URL of callback API (specifically **code**)

Name	Headers	Payload	Preview	Response	Initiator	Timing
authenticate?session...						
callback?session_state...						
favicon.ico						
		▼ Query String Parameters				
		session_state		d95ee64f-a669-4a6e-b67a-7dd1108ad7e3		
		iss		http://localhost:8180/realms/eazybankdev		
		code		dc273d63-da52-48db-b260-2a9c078de84a.d95ee64f-a669-4a6e-b67a-7dd1108ad7e3.173cbd05-88bb-4454-8a63-6aa0b7c38c11		

Note:

- The state value isn't strictly necessary here since the PKCE parameters provide CSRF protection themselves.
- In practice, if we're sure the OAuth server supports PKCE, we can use the state parameter for application state instead of using it for CSRF protection.

Step 7: Exchange the Authorization code

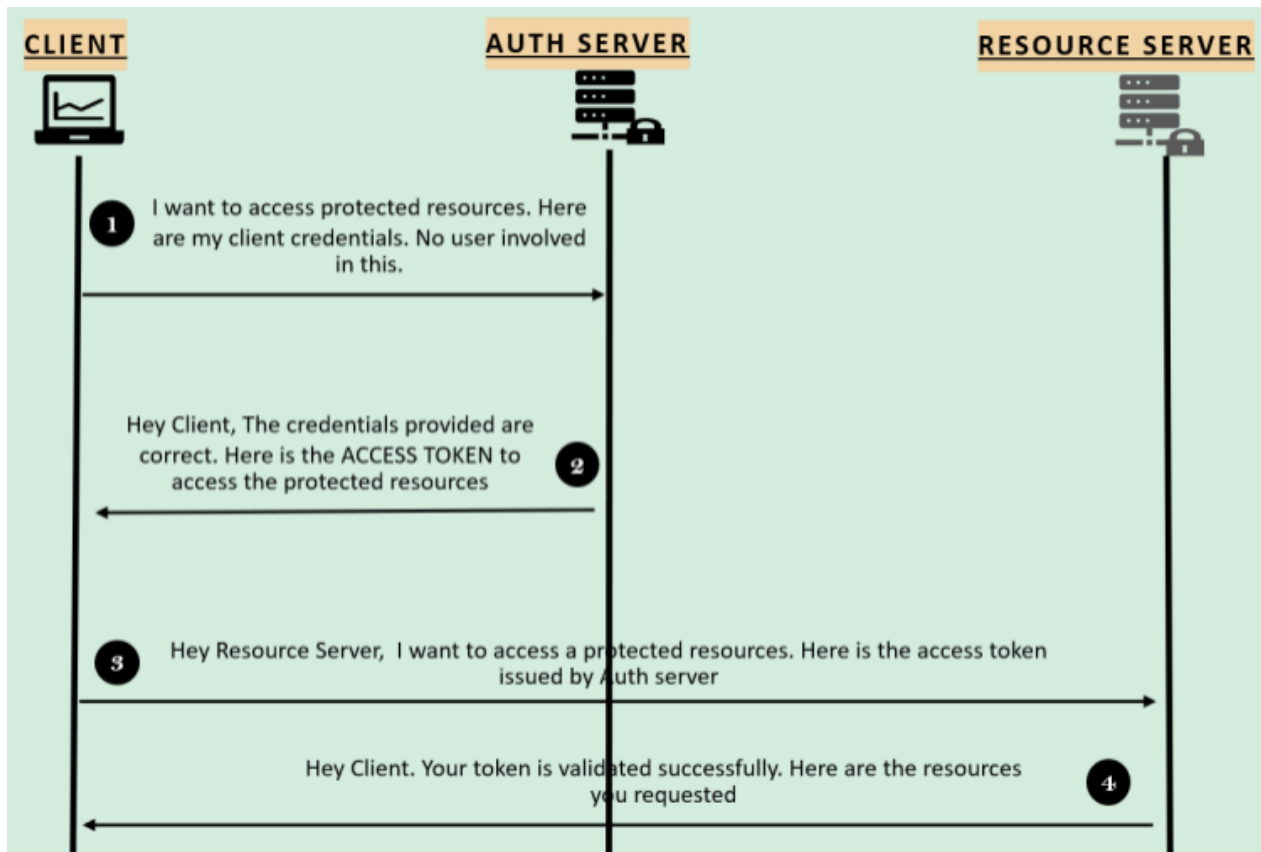
- The client will build a POST request to the token endpoint with below parameters:

```
▼ POST http://localhost:8180/realms/eazybankdev/protocol/openid-connect/token
  ► Network
  ► Request Headers
  ▼ Request Body
    grant_type: "authorization_code"
    code: "dc273d63-da52-48db-b260-2a9c078de84a.d95ee64f-a669-4a6e-b67a-7dd1108ad7e3.173cbd05-88bb-4454-8a63-6aa0b7c38c11"
    redirect_uri: "https://oauth.pstmn.io/v1/callback"
    code_verifier: "w5xI79emoXFX5MjM6jzJ9ZlLF7KAcHPKu-dzCz_na8"
    client_id: "eazybankpkce"
  ► Response Headers
```

- We'll need to send the `code_verifier` along with the token request. The Authorization server will check whether the verifier matches the challenge that was used in the Authorization request. This ensures that a malicious party that intercepted the Authorization code will not be able to use it.

[illegible]

Client Credential Grant Type



Step 1: Client Registration – Post registration, we will have ClientID, ClientSecret

eazybankclientcredential OpenID Connect

Clients are applications and services that can request authentication

Settings

Keys

Credentials

Roles

Client scopes

General settings

Client ID * ?

Name ?

Description ?

Always display in UI ? ☐ Off

Capability config

Client authentication ? ☒ On

Authorization ? ☐ Off

Authentication flow

☐ Standard flow ?

☐ Direct access grants ?

☐ Implicit flow ?

☒ Service accounts roles ?

☐ Standard Token Exchange ?

☐ OAuth 2.0 Device Authorization Grant ?

☐ OIDC CIBA Grant ?

PKCE Method ?

Client Id – eazybankclientcredential

Client Secret - d409AMxaAsOAsDUEEzGAnL72fjqslCtS

eazybankclientcredential OpenID Connect

Clients are applications and services that can request authentication of

Settings	Keys	Credentials	Roles	Client scopes
----------	------	-------------	-------	---------------

Client Authenticator	Client Id and Secret
Save	
Client Secret	d409AMxaAsOAsDUEEzGAnL72fjqsICtS

Step 2: Get token URI from below endpoint (under Realm setting – make sure you’ve selected the correct realm i.e., eazybankdev

OpenID Configuration - <http://localhost:8180/realms/eazybankdev/.well-known/openid-configuration>

Events

Configure

Realm settings

Authentication

Endpoints

OpenID Endpoint Configuration

SAML 2.0 Identity Provider Metadata

Save

Revert

Step 3: Add all these configurations to Postman to generate Access Token

Client Id, Client Secret

Access Token URL - <http://localhost:8180/realms/eazybankdev/protocol/openid-connect/token>

Params	Authorization	Headers (9)	Body	Scripts	Settings
Auth Type OAuth 2.0 The authorization data will be automatically generated when you send the request. Learn more about OAuth 2.0 authorization. Add authorization data to Request Hea...			Grant type Client Credentials Access Token URL http://localhost:8180/realms/eazybankdev/... Client ID eazybankclientcredential Client Secret 09AMxaAsOAsDUEEzGAnL72fjqsICtS Scope openid email Client Authentication Send client credentials in body		

Step 3: Get Access Token (Exchange the Secret)

```
▼ POST http://localhost:8180/realms/eazybankdev/protocol/openid-connect/token
  ► Network
  ► Request Headers
  ▼ Request Body
    grant_type: "client_credentials"
    scope: "openid email"
    client_id: "eazybankclientcredential"
    client_secret: "d409AMxaAsOAsDUEEzGAnL72fjqSICTs"
  ► Response Headers
  ► Response Body ↗
```

< Response Body

{ } JSON ▾

```
1 {
2   "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXZWQ0aWI6ICJhcGE4dFpVa3RVRVhWRWVoYyJleHAiOiJlZ3NTM5NDgyNjksIm1hdCI6MTc1Mzk0Nzk2OSwianRpIjoidHJydGNjOjVmdZDVhZGVlTA20TQtZD
3   "expires_in": 3600,
4   "refresh_expires_in": 0,
5   "token_type": "Bearer",
6   "id_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXZWQ0aWI6ICJhcGE4dFpVa3RVRVhWRWVoY1Axw
7   "not-before-policy": 0,
8   "scope": "openid profile email"
9 }
```

Note:

- Refer <https://www.keycloak.org/documentation> <https://www.keycloak.org/docs-api/latest/rest-api/index.html> for more details on how to configure users through APIs
- We can refer <https://www.keycloak.org/guides> for setting up Keycloak.
- We can use <https://oauth.net/2/> , <https://www.oauth.com/> , <https://www.oauth.com/playground/> to explore all the in-depth OAuth2 details & grant types.